Бойер-Мур

Основная идея алгоритма Бойера-Мура заключается в том, что паттерн прикладывается к тексту и начинает проверяться справа налево. В случае несовпадение существует две эвристики, которые определяют на сколько позиций необходимо передвинуть паттерн:

1. Правило плохого символа - ППС (сильное правило плохого символа)
2. Правило хорошего суффикса - ПХС

Правило плохого символа

Идея заключается в том, чтобы для каждого символа паттерна хранить индекс его самого правого вхождения.

Алгоритм препроцессинга паттерна по ППС:

1. Выделить хеш-мапу, где ключом будем символ (char), а значением будет его самый правый индекс в этом паттерне.
2. Начать обход паттерна слева направо
   - Если символ α в позиции i не находится в хеш-мапе, то записать в хеш мапу пару α: i.
   - Если символ α в позиции i уже встречался в хеш-мапе, то его старый индекс j гарантированно меньше i. Чтобы удовлетворить условию, что нужно самое правое вхождение, перезапишем i вместо j для ключа α.

Сложность: $O(m)$, где m - длина паттерна
Доказательство: Один проход по паттерну слева направо без вложенных циклов.

Пример препроцессинга:

Алгоритм Бойера-Мура с использованием ППС:

1. Сделать препроцессинг паттерна по ППС.
2. К левому концу текста приложить левый конец паттерна.
3. Начать сравнивать символы справа налево: T_m ? P_m, T_{m-1} ? P_{m-1}, ... , T_1 ? P_1.
4. Если при сравнение символов на позиции j текста и позиции i паттерна произошло несовпадение (т.е. $T_j \neq P_i$), то обратится в хеш-мапу по ключу T_j и получить индекс k.
   - Если $k >= i$, то паттерн сдвигается на 1 позицию правее. Возврат к п.3.
   - Если $k < i$, то паттерн сдвигается так, чтобы $T_j = P_k$ (сдвиг на $i - k$ позиций). Возврат к п.3.

- Если k не существует, то паттерн сдвигается так, чтобы T_{j+1} и P_{i+1} были на одном уровне (сдвиг на $i+1$ позицию). Возврат к п.3.

5. Если паттерн был полностью сравнён с текстом и несовпадений не случилось, то найдено вхождение паттерна P в текст T. Паттерн сдвигается на $|P| = m$ позиций.

Сложность: $O(n \cdot m)$ в худшем случае и $O(n+m)$ в среднем , где m - длина паттерна, n - длина текста.
Доказательство: Препроцессинг паттерна занимает $O(m)$ времени, т.к. происходит один проход по строке без вложенных циклов. Поиск в среднем работает за $O(n)$, потому что оказывается, что символ текста либо не встречается в паттерне, либо позволяет за одно сравнение сделать сдвиг больший чем 1. Следовательно, получается, что каждый символ тексте в среднем будет сравниваться с паттерном 1 раз, тогда для n символов текста сложность окажется линейной. В худшем же случае в алгоритм будет подана строка состоящая из одного и того же символа, а в паттерне на всех позициях кроме первой будет тот же символ. Тогда на для каждого символа текста придётся делать ровно m сравнений с паттерном, что для n символов превратится в $n \cdot m$.

Пример применения БМ с ППС:

Пример применения БМ с ППС для худшего случая:

Сильное правило плохого символа

Идея заключается в том, чтобы для каждого символа паттерна хранить отсортированный массив индексов всех его вхождений в паттерн.

Алгоритм препроцессинга паттерна по СППС:

1. Выделить хеш-мапу, где ключом будем символ (char), а значением будет отсортированный массив его индексов.
2. Начать обход паттерна слева направо
   - Если символ α в позиции i не находится в хеш-мапе, то записать в хеш мапу пару $\alpha : [i]$.
   - Если символ α в позиции i уже встречался в хеш-мапе, то в массив индексов дописать i: α : = $[j, \ldots, i]$

Сложность: $O(m)$, где m - длина паттерна
Доказательство: Один проход по паттерну слева направо без вложенных циклов.

Пример препроцессинга:

Алгоритм Бойера-Мура с использованием СППС:

1. Сделать препроцессинг паттерна по СППС.
2. К левому концу текста приложить левый конец паттерна.
3. Начать сравнивать символы справа налево: T_m ? P_m, T_{m-1} ? P_{m-1}, ... , T_1 ? P_1.
4. Если при сравнение символов на позиции j текста и позиции i паттерна произошло несовпадение (т.е. $T_j \neq P_i$), то обратится в хеш-мапу по ключу T_j и получить массив индексов A.
   - Если массив индексов пустой, то паттерн сдвигается так, чтобы T_{j+1} и P_{i+1} были на одном уровне (сдвиг на $i+1$ позицию). Возврат к п.3.
   - Если $\forall k \in A\ k >= i$, то паттерн сдвигается на 1 позицию правее. Возврат к п.3.
   - Если $\exists k \in A : k < i$, то для $k = max(k_l : k_l < i)$ паттерн сдвигается так, чтобы $T_j = P_k$ (сдвиг на $i - k$ позиций). Возврат к п.3.
5. Если паттерн был полностью сопоставлен с текстом и несовпадений не случилось, то найдено вхождение паттерна P в текст T. Паттерн сдвигается на $|P| = m$ позиций.

Сложность: $O(n \cdot m)$ в худшем случае и $O(n + m)$ в среднем. Доказательство аналогичное ППС.

Пример применения БМ с СППС:

Пример примнения БМ с СППС для худшего случая:

Правило хорошего суффикса

Идея заключается в том, что мы приложили паттерн к тексту и у нас совпало l символов, т.е. суффикс α, $|\alpha| = l$. При этом нам известно, что левее в паттерне подстрока α встречается ещё хотя бы 1 раз. Тогда мы можем сдвинуть паттерн так, чтобы это более левое вхождение α в паттерн встало под текущее α в тексте.

Алгоритм препроцессинга паттерна по ПХС

1. Для паттерна P реверсивно записать P_{rev}.
2. Для P_{rev} посчитать Z-функцию.
3. Выделить N-массив длины $|P| = m$, заполнить его нулями.
4. Начать обход слева направо для Z-массива.
   - Если $Z_i \neq 0$ пользуемся формулой $N_j = Z_i$, где $j = m - i + 1$.
5. Выделить массив L' длины m, заполнить его нулями.
6. Выделить массив l' длины m, заполнить его нулями.
7. Завести переменную `max_prefix_len` $= 0$. Она будет хранить длину максимального префикса паттерна, который одновременно является его суффиксом.
8. Начать обход слева направо для N-массива.
   - Если $N_i = i$, то `max_prefix_len` $= i$

- $l'_j =$ `max_prefix_len`, где $j = m - i + 1$.
- Если $N_i \neq 0$, то пользуемся формулой $L'_j = i$, где $j = m - N_i + 1$.

9. Возвращаем L' и l' массивы.

Примечание 1: В N-массиве хранятся длины подстрок, которые оканчиваются в этом индексе и являются суффиксом паттерна.
Примечание 2: В L'-массиве хранятся индексы подстрок, которые в этой позиции заканчиваются и являются суффиксами паттерна.
Примечание 3: В L'-массиве хранятся индексы самых правых подстрок, удовлетворяющих *Примечанию 2*, так как при проходе слева направо новые значения перезаписывают старые, находившиеся левее.
Примечание 4: В l'-массиве для каждой позиции i хранится **длина** максимального префикса паттерна P, который одновременно является суффиксом совпавшей в тексте подстроки (длина которой равна $m - i$.

Сложность: $O(m)$, где m - длина паттерна
Доказательство: Ревёрс строки длины m выполняется за линию. Подсчёт Z-фукнции для строки длины m выполняется за линию. Подсчёт N-функции через Z-функцию для строки длины m выполняется за линию. Подсчёт L'-функции через N-фукнцию для строки длины m выполняется за линию. Итого препроцессинг выполняется за $m + m + m + m = 4m$, $O(4m) \approx O(m)$. **ЧТД.**

Пример препроцессинга:

Алгоритм Бойера-Мура с использованием ПХС:

1. Выполнить препроцессинг по ПХС - получить L'-массив.
2. К левому концу текста приложить левый конец паттерна.
3. Начать сравнивать символы справа налево.
4. Если при сравнение символов на позиции j текста и позиции i паттерна произошло несовпадение (т.е. $T_j \neq P_i$), то мы вычисляем сдвиг правого конца приложения паттерна в тексте (r) на основе суффикса, который мы уже успели успешно сопоставить (он начинается с позиции $i + 1$):
   - Если $L'_{i+1} > 0$, то `shift` = $m - L'_{i+1}$.
   - Если $L'_{i+1} = 0$, то `shift` = $m - l'_{i+1}$.
   - Обновляем позицию правого конца приложения паттерна: r += 'shift', и возвращаемся к п. 3.
5. Если паттерн был полностью сопоставлен с текстом и несовпадений не случилось, то найдено вхождение паттерна P в текст T. Паттерн сдвигается на $|P| = m$ позиций.

Сложность: $O(n+m)$

Доказательство: Препроцессинг занимает $O(m)$. Используя массивы L' и l', при любом совпадении суффикса длины k гарантированно сдвигаем паттерн вперед на k символов. В худшем случае каждый символ при ПХС сравнивается не более $7n$ раз, $O(7n) \approx O(n)$. Итого суммарная сложность алгоритма составляет $O(n+m)$. **ЧТД.**

Пример применения БМ с ПХС:

Совместное использование СППС и ПХС:

1. Выполнить препроцессинг по СППС.
2. Выполнить препроцессинг по ПХС.
3. К левому концу текста приложить левый конец паттерна.
4. Начать сравнивать символы справа налево.
5. Если при сравнение символов на позиции j текста и позиции i паттерна произошло несовпадение (т.е. $T_j \neq P_i$), то выполнить сдвиг на max(СППС, ПХС, 1). Возвращаемся к п.3.
6. Если паттерн был полностью сопоставлен с текстом и несовпадений не случилось, то найдено вхождение паттерна P в текст T. Паттерн сдвигается на $|P| = m$ позиций.

Сложность: $O(n+m)$

Доказательство: аналогично ПХС, т.к. это правило сильнее чем СППС и статистически выполняется чаще.

Пример применения БМ при совместном использовании СППС и ПХС: